Blockchain Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

MetaMask is a **cryptocurrency wallet and gateway** to blockchain applications. It allows users to store, send, and receive Ether or tokens, and connect to decentralized applications (DApps) directly from a web browser. MetaMask acts as a **bridge** between a user's browser and the Ethereum blockchain (or other compatible networks).

Features**:**

- Easy management of multiple blockchain accounts
- Secure private key storage
- Integration with test networks like **Sepolia**, **Linea**, or **RSK**
- Interaction with DApps and smart contracts via **Remix IDE**

Testnet:

A Testnet is a blockchain network used for testing and development without using real cryptocurrency. It mimics the main Ethereum network but uses test Ether (fake ETH). Common testnets include Sepolia Testnet, Linea Testnet, and RSK Testnet.

Advantages:

- No real money loss during testing
- Safe environment for developers
- Helps test smart contracts and transactions before mainnet deployment

Steps to Connect MetaMask with Remix IDE for Performing Transactions:

1. Set Up MetaMask:
    - Install the MetaMask extension on your browser.
    - Create or import a wallet account using a recovery phrase.
2. Add a Test Network:
    - Open MetaMask → Settings → Networks → Add Network.
    - Enter details for the desired Testnet (e.g., Sepolia or RSK).
    - Example RPC URL: https://sepolia.infura.io/v3/<your_project_id>
3. Fund the Wallet:
    - Use a faucet to receive free test Ether (e.g., 0.5 ETH/day).
    - Check the balance in MetaMask.
4. Connect MetaMask with Remix IDE:
    - Open Remix IDE.
    - In the Deploy & Run Transactions tab, choose Injected Web3 as the environment.
    - Approve connection in MetaMask.

Create and Deploy a Simple Solidity Smart Contract:

Example:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Counter {
   uint public count;
   function increment() public {
      count += 1;
   }
   function getCount() public view returns (uint) {
      return count;
   }
}
```

5. Create and Deploy a Simple Solidity Smart Contract:

- Compile and deploy the contract using Remix.
- Confirm the transaction in MetaMask.

6. Check Transaction Details:
   - View the transaction status on RSK Explorer or Etherscan (for Sepolia).
7. Interact with the Contract:
   - Use Remix to call contract functions (increment(), getCount()), and observe state changes and transaction confirmations.

Implementation:

- FarmerContract.sol:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
contract FarmerContract {
   struct Batch {
      uint256 batchID;
      string cropName;
      uint256 quantity;
      uint256 timestamp;
      address farmer;
   }
   mapping(uint256 => Batch) public batches;
   event BatchCreated(
      uint256 batchID,
      string cropName,
      uint256 quantity,
      uint256 timestamp,
      address farmer
   );
   function addBatch(uint256 _batchID, string
memory _cropName, uint256 _quantity) public {
      require(_batchID != 0, "Invalid batch ID");
      require(batches[_batchID].batchID == 0, "Batch
already exists!");

      batches[_batchID] = Batch({
         batchID: _batchID,
         cropName: _cropName,
         quantity: _quantity,
         timestamp: block.timestamp,
         farmer: msg.sender
      });
      emit BatchCreated(_batchID, _cropName,
_quantity, block.timestamp, msg.sender);
   }
   function getBatch(uint256 _batchID) public view
returns (
      uint256,
      string memory,
      uint256,
      uint256,
      address
   ) {
      Batch memory b = batches[_batchID];
      require(b.batchID != 0, "Batch does not exist!");
      return (b.batchID, b.cropName, b.quantity,
b.timestamp, b.farmer);
   }
}
```

- DistributorContract.sol:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
interface IFarmerContract {
   function getBatch(uint256 _batchID) external view
returns (
      uint256,
      string memory,
      uint256,
      uint256,
      address
   );
}
contract DistributorContract {
   struct ReceivedBatch {
      uint256 batchID;
      string cropName;
      uint256 quantity;
      uint256 timestamp;
      address farmer;
      address distributor;
   }
   mapping(uint256 => ReceivedBatch) public
receivedBatches;
   IFarmerContract public farmerContract;
   event BatchReceived(
      uint256 batchID,
      string cropName,
      uint256 quantity,
      address farmer,
      address distributor,
      uint256 timestamp
   );
   constructor(address _farmerContractAddress) {
      farmerContract =
IFarmerContract(_farmerContractAddress);
   }
```

```
    function receiveBatch(uint256 _batchID) public {
        (
            uint256 id,
            string memory cropName,
            uint256 quantity,
            , // ignore farmer timestamp
            address farmer
        ) = farmerContract.getBatch(_batchID);
        require(id != 0, "Batch does not exist in
FarmerContract");
        require(receivedBatches[_batchID].batchID ==
0, "Batch already received!");
        receivedBatches[_batchID] = ReceivedBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
            farmer: farmer,
            distributor: msg.sender
        });
        emit BatchReceived(id, cropName, quantity,
farmer, msg.sender, block.timestamp);
    }
```

- RetailerContract.sol:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
interface IDistributorContract {
    function getReceivedBatch(uint256 _batchID)
external view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address
    );
}
contract RetailerContract {
    struct RetailBatch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
        address distributor;
        address retailer;
    }
    mapping(uint256 => RetailBatch) public
retailBatches;
    IDistributorContract public distributorContract;
    event BatchReceivedAtRetailer(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        address farmer,
        address distributor,
        address retailer,
        uint256 timestamp
    );
    constructor(address _distributorContractAddress) {
        distributorContract =
IDistributorContract(_distributorContractAddress);
    }

    function receiveFromDistributor(uint256
_batchID) public {
        (
            uint256 id,
            string memory cropName,
            uint256 quantity,
            , // ignore distributor timestamp
            address farmer,
            address distributor
        ) =
distributorContract.getReceivedBatch(_batchID);
        require(id != 0, "Batch does not exist in
DistributorContract");
        require(retailBatches[_batchID].batchID == 0,
"Batch already received at Retailer!");
        retailBatches[_batchID] = RetailBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
```

```
    function getReceivedBatch(uint256 _batchID)
public view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address
    ) {
        ReceivedBatch memory rb =
receivedBatches[_batchID];
        require(rb.batchID != 0, "Batch not received!");
        return (
            rb.batchID,
            rb.cropName,
            rb.quantity,
            rb.timestamp,
            rb.farmer,
            rb.distributor
        );
    }
}
```

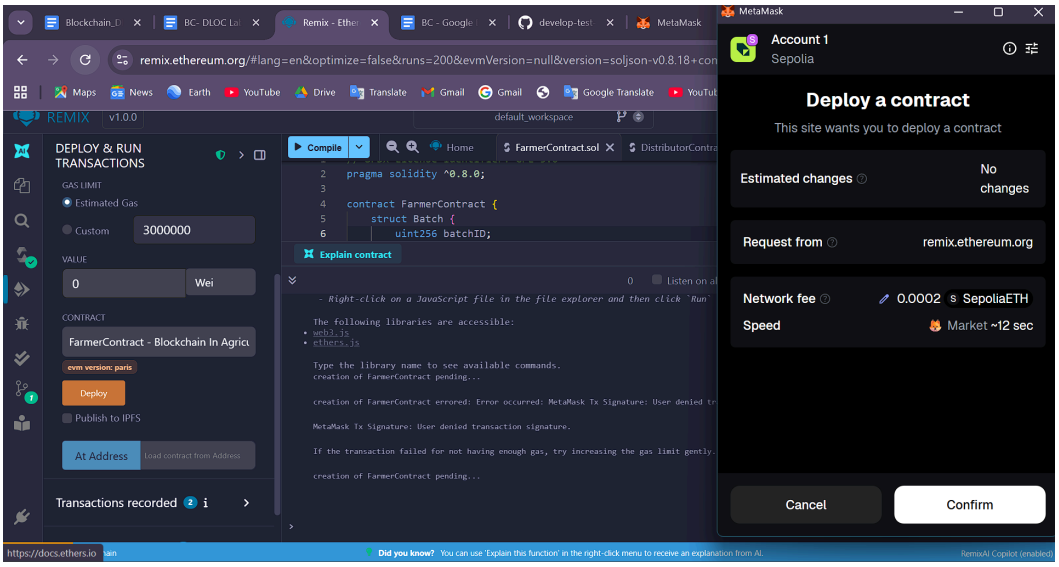
```
        farmer: farmer,
        distributor: distributor,
        retailer: msg.sender
    });
    emit BatchReceivedAtRetailer(
        id,
        cropName,
        quantity,
        farmer,
        distributor,
        msg.sender,
        block.timestamp
    );
  }
  function getRetailBatch(uint256 _batchID) public
view returns (
    uint256,
    string memory,
    uint256,
    uint256,
    address,
    address,
    address
  ) {
    RetailBatch memory rb =
retailBatches[_batchID];
    require(rb.batchID != 0, "Batch not received at
retailer!");

    return (
      rb.batchID,
      rb.cropName,
      rb.quantity,
      rb.timestamp,
      rb.farmer,
      rb.distributor,
      rb.retailer
    );
  }
```

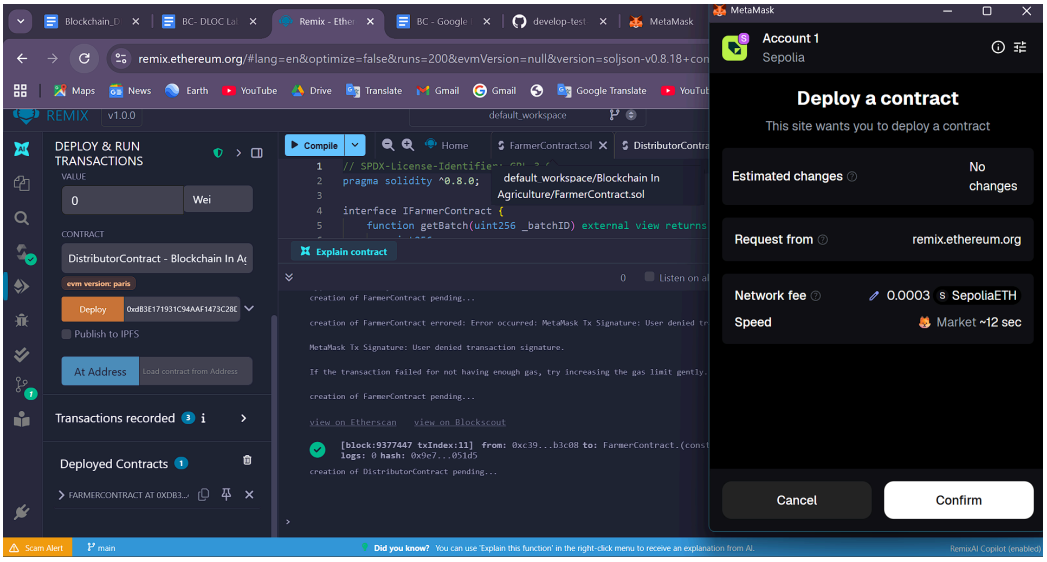
- <u>CustomerContract.sol:</u>

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
interface IRetailerContract {
  function getRetailBatch(uint256 _batchID)
external view returns (
    uint256,
    string memory,
    uint256,
    uint256,
    address,
    address,
    address
  );
}
contract CustomerContract {
  struct PurchasedBatch {
    uint256 batchID;
    string cropName;
    uint256 quantity;
    uint256 timestamp;
    address farmer;
    address distributor;
    address retailer;
    address customer;
  }
  mapping(uint256 => PurchasedBatch) public
purchasedBatches;
  IRetailerContract public retailerContract;
  event BatchPurchased(
    uint256 batchID,
    string cropName,
    uint256 quantity,
    address farmer,
    address distributor,
    address retailer,
    address customer,
    uint256 timestamp
  );
  constructor(address _retailerContractAddress) {
    retailerContract =
IRetailerContract(_retailerContractAddress);
  }
  function purchaseFromRetailer(uint256 _batchID)
public {
    (
      uint256 id,
      string memory cropName,
      uint256 quantity,
      , // ignore retailer timestamp
      address farmer,
      address distributor,
      address retailer
    ) = retailerContract.getRetailBatch(_batchID);
    require(id != 0, "Batch does not exist in
RetailerContract");
    require(purchasedBatches[_batchID].batchID ==
0, "Batch already purchased!");
    purchasedBatches[_batchID] =
PurchasedBatch({
      batchID: id,
      cropName: cropName,
      quantity: quantity,
      timestamp: block.timestamp,
      farmer: farmer,
      distributor: distributor,
      retailer: retailer,
```

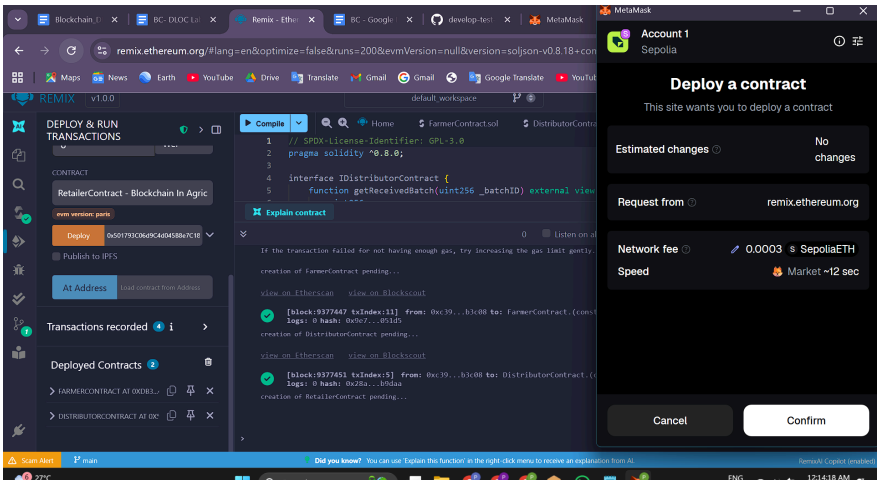
```
        customer: msg.sender
    });
    emit BatchPurchased(
        id,
        cropName,
        quantity,
        farmer,
        distributor,
        retailer,
        msg.sender,
        block.timestamp
    );
  }
  function getPurchasedBatch(uint256 _batchID)
public view returns (
    uint256,
    string memory,
    uint256,
    uint256,
    address,
    address,
    address,
    address
  ) {
    PurchasedBatch memory pb =
purchasedBatches[_batchID];
    require(pb.batchID != 0, "Batch not
purchased!");
    return (
      pb.batchID,
      pb.cropName,
      pb.quantity,
      pb.timestamp,
      pb.farmer,
      pb.distributor,
      pb.retailer,
      pb.customer
    );
  }
}
```

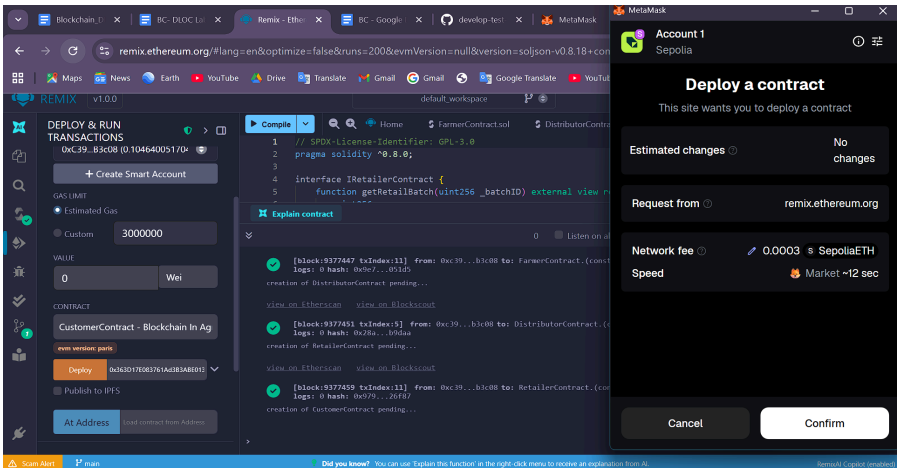
Output:

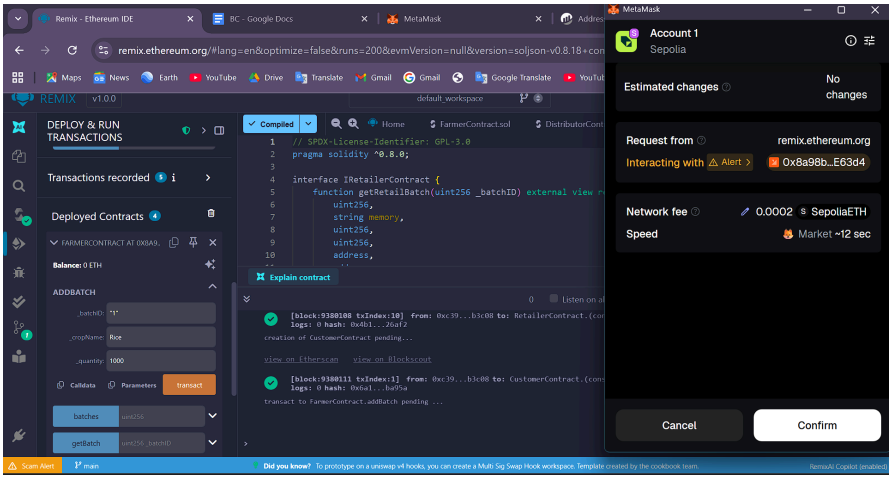


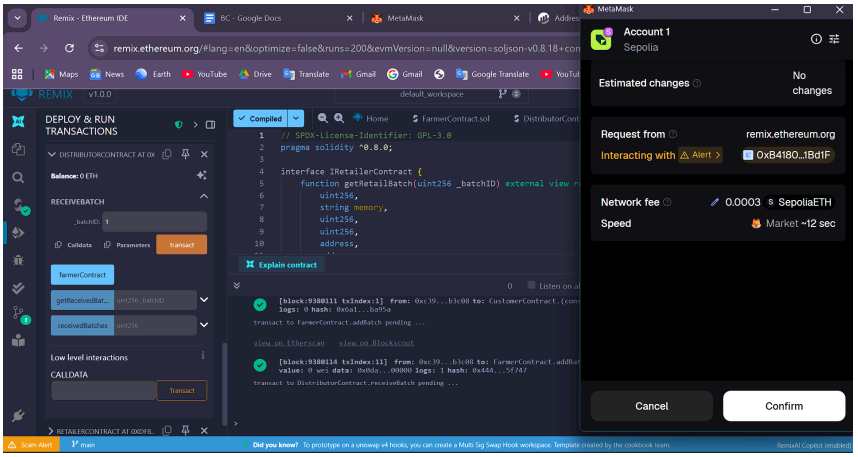


Prajyot Shinde *D20A* *Roll No.57*

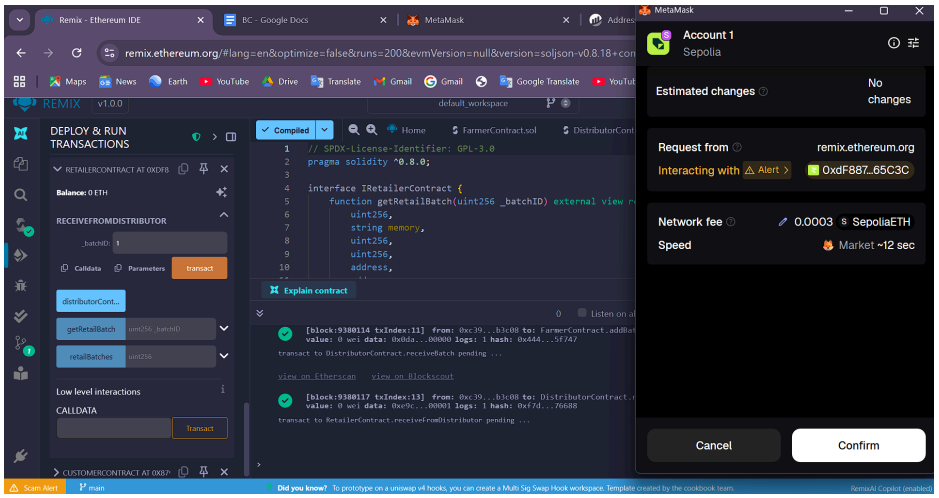


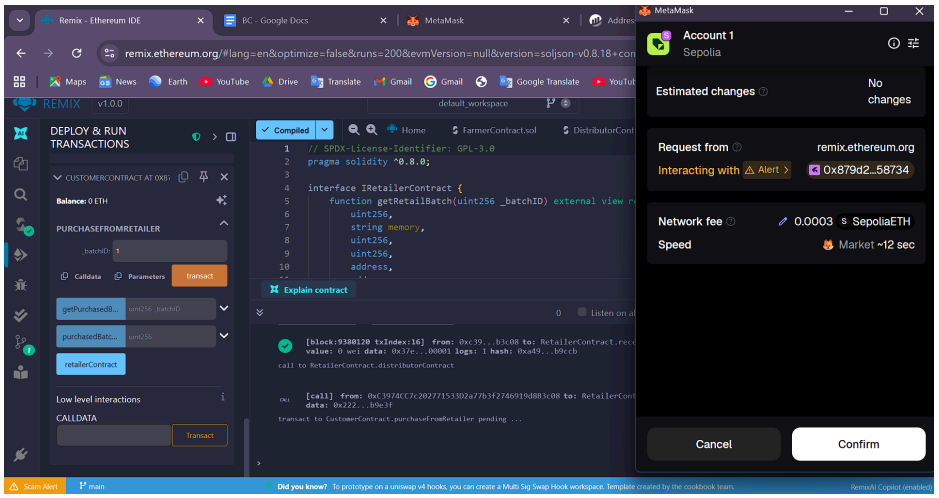


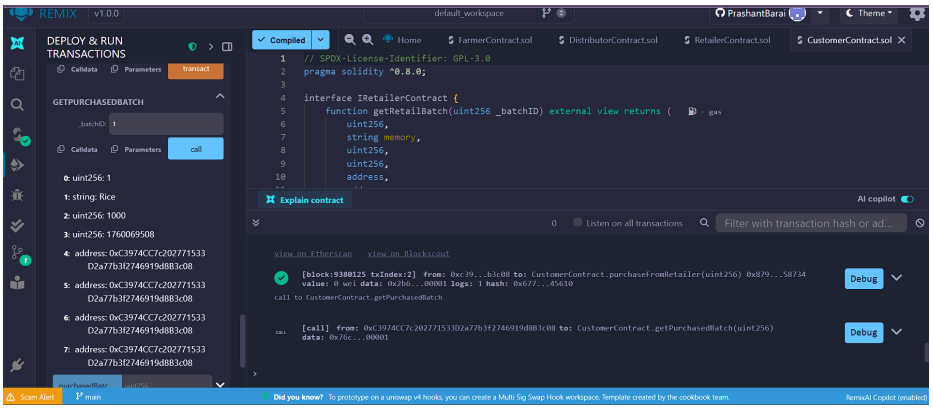


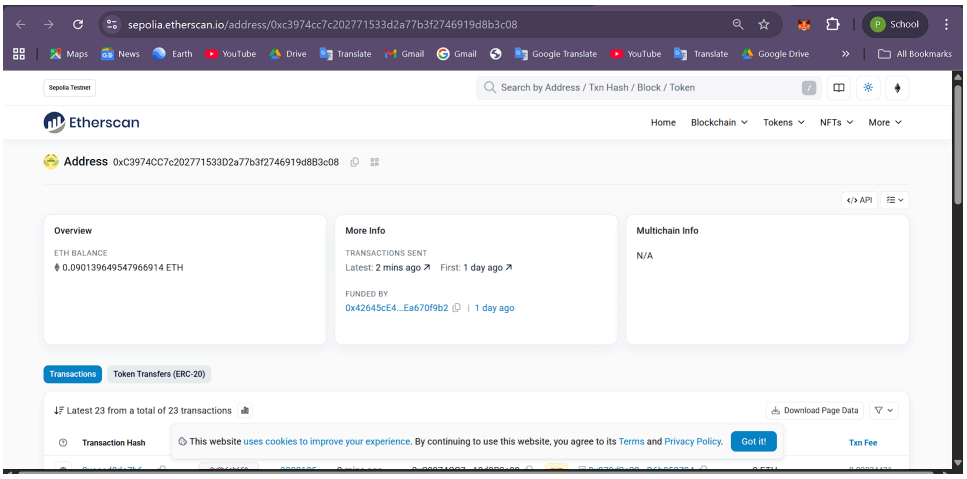


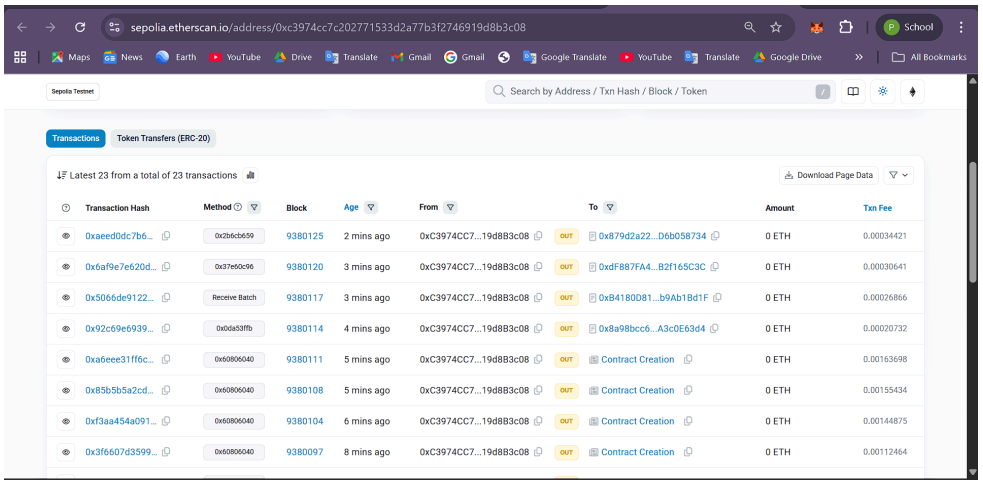
Prajyot Shinde *D20A* *Roll No.57*

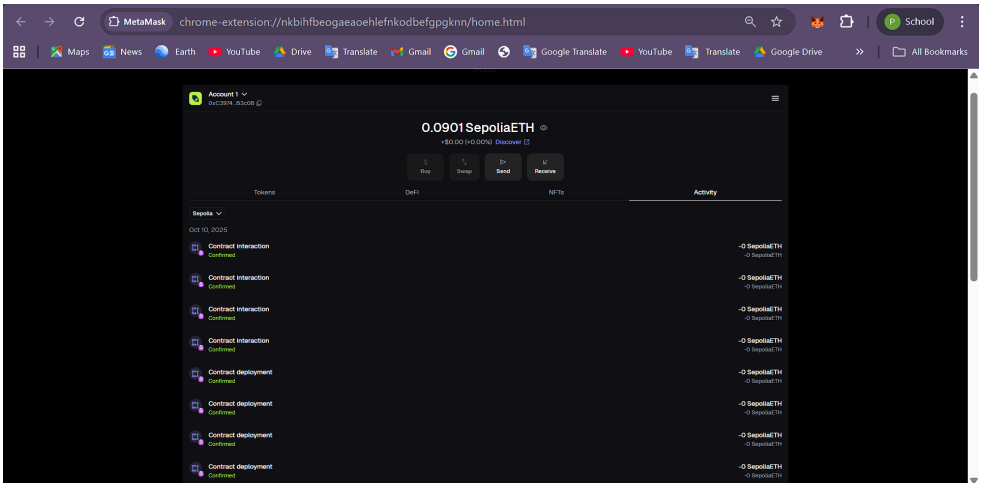












Conclusion:

Using MetaMask in combination with Remix IDE offers a highly practical and hands-on approach to testing and deploying smart contracts on various blockchain test networks. This integration allows developers to move beyond purely theoretical coding and gain real-world experience by interacting directly with decentralized networks in a controlled setting. By working with testnets, developers can simulate actual deployment scenarios, observe how transactions are processed, and understand important concepts such as gas fees, confirmations, and network latency without the financial risks associated with mainnet deployments.

Moreover, this setup provides valuable insight into how wallet integrations function, enabling developers to better grasp how users will interact with decentralized applications (dApps) in real scenarios. It also helps in identifying and debugging potential issues early in the development cycle, improving the overall reliability and security of smart contracts. Ultimately, combining MetaMask with Remix IDE creates a safe, efficient, and educational environment that bridges the gap between development and real blockchain usage, making it an essential workflow for both beginners and experienced blockchain developers.